Comparación de algoritmos de ordenamiento en C++: MergeSort vs QuickSort

Comparison of sorting algorithms in C++: MergeSort vs. QuickSort

Joel Joshet Gomez Chachaque

jgomezc@unjbg.edu.pe

Facultad de Ingeniería, Universidad Nacional Jorge Basadre Grohmann, Escuela Profesional de Ingeniería en Informática y Sistemas, Tacna, Perú
ORCID: 0009-0003-9271-4501

Resumen: *El objetivo es comparar el ordenamiento por el método MergeSort y QuickSort midiendo su tiempo de ejecución, numero de comparaciones, intercambios, profundidad de recursión, memoria auxiliar y estabilidad. Para ello se implementaron ambos algoritmos en C++ y se ejecutaron cinco patrones de entrada (aleatorio, ascendente, descendente, casi ordenado y con duplicados), con 1 000, 10 000 y 100 000 datos y con semillas fijas; se calculó estadística descriptiva. Como resultado en este experimento y con estas implementaciones, QuickSort fue más rápido que MergeSort en todos los escenarios medidos. Eso no significa que QuickSort siempre sea superior en cualquier situación. Para concluir, en las implementaciones utilizadas, QuickSort tuvo un mejor rendimiento temporal en el experimento realizado mostrando una ventaja especialmente marcada con arreglos ordenados de forma ascendente y descendente. Además, el número de comparaciones no fue suficiente para determinar qué algoritmo era más rápido. Aunque MergeSort realizó menos comparaciones en algunos escenarios, QuickSort obtuvo menores tiempos y contabilizó menos movimientos en ciertos casos.*

Palabras Clave: MergeSort, QuickSort, ordenamiento, C++

Abstract: *The objective is to compare the MergeSort and QuickSort sorting methods by measuring their execution time, number of comparisons, swaps, recursion depth, auxiliary memory usage, and stability. To this end, both algorithms were implemented in C++ and tested against five input patterns (random, ascending, descending, nearly sorted, and containing duplicates), using datasets of 1,000, 10,000, and 100,000 elements with fixed seeds; descriptive statistics were then calculated. In this experiment—given these specific implementations—QuickSort proved faster than MergeSort across all measured scenarios. This does not imply that QuickSort is invariably superior in every situation. In conclusion, within the scope of the implementations used, QuickSort demonstrated better temporal performance, showing a particularly significant advantage with arrays that were already sorted in ascending or descending order. Furthermore, the number of comparisons alone was not a sufficient metric to determine which algorithm was faster; although MergeSort performed fewer comparisons in some scenarios, QuickSort achieved shorter execution times and, in certain cases, required fewer data movements.*

Keywords: MergeSort, QuickSort, sorting, C++

1. Introducción

Los algoritmos de ordenamiento constituyen herramientas esenciales en programación, pues permiten organizar los datos siguiendo un criterio específico, facilitando así su búsqueda, análisis y procesamiento. Su relevancia se amplifica en el manejo de grandes volúmenes de información, ya que la eficiencia del algoritmo empleado puede afectar al tiempo y recursos requeridos para finalizar una tarea. Por eso, resulta relevante estudiar el rendimiento de distintos métodos de ordenamiento, para entender cómo se comportan y decidir en qué situaciones resultan más convenientes.

Los algoritmos más conocidos son MergeSort y QuickSort, y ambos utilizan estrategias diferentes para ordenar una secuencia de elementos. MergeSort divide los datos en partes más pequeñas, los ordena y luego los combina, mientras que QuickSort utiliza un elemento pivote para dividir el arreglo en particiones. Ambos emplean la estrategia de divide y vencerás, pero las diferencias en su funcionamiento pueden provocar que el número de operaciones y el tiempo de ejecución varíen en función del tamaño y de la distribución inicial de los datos.

En este sentido, la presente investigación pretende comparar de manera experimental el rendimiento de MergeSort y QuickSort, a través de implementaciones desarrolladas en C++. Para tal fin se analizan diferentes tamaños de arreglos y patrones de entrada considerando métricas como el tiempo de ejecución, el número de comparaciones, los movimientos, los intercambios y la profundidad máxima de recursión. También se realizan varias repeticiones, con el fin de obtener mediciones que permitan comparar el comportamiento de ambos algoritmos en condiciones equivalentes.

Por último, los resultados obtenidos hacen posible comparar las diferencias de desempeño entre los algoritmos y relacionarlas con las operaciones que efectúan durante el proceso de ordenamiento. Así, el estudio pretende aportar una comprensión práctica de sus características y limitaciones, entendiendo que el desempeño observado depende tanto de la estrategia del algoritmo como de su implementación y de las condiciones del experimento.

2. Método

2.1. Algoritmos:

Se compararon dos algoritmos de ordenamiento: MergeSort y QuickSort, ambos implementados en el lenguaje de programación C++. Los dos emplean la estrategia de divide y vencerás, pero presentan diferencias en la manera de organizar los elementos.
MergeSort divide recursivamente el arreglo en subarreglos hasta alcanzar partes de un solo elemento. Posteriormente, combina estas partes en orden, utilizando un arreglo auxiliar durante la combinación. Por su parte, QuickSort selecciona un pivote y reorganiza los elementos mediante un procedimiento de partición, de manera que los elementos queden distribuidos a ambos lados del pivote según su valor. En esta

implementación se utilizó el elemento central del arreglo como pivote y el esquema de partición de Hoare.

2.2. Entorno de reproducción:

Los algoritmos se desarrollaron en C++ y se compilaron utilizando el compilador g++ en Visual Studio Code, en un entorno Windows 11.

El equipo utilizado fue una Laptop ASUS TUF Gaming A15 con procesador AMD Ryzen 7 a 3.10Ghz, 16 GB de RAM y 512 GB de almacenamiento.

El tiempo de ejecución se midió mediante std::chrono::steady_clock, que permite registrar intervalos de tiempo. La medición comenzó inmediatamente antes de ejecutar cada algoritmo y terminó al finalizar el ordenamiento. Se excluyeron de este intervalo la generación de los datos, la copia inicial del arreglo, la validación del resultado y la escritura de los registros en el archivo CSV.

2.3. Generación de datos:

Se evaluaron tres tamaños de entrada: 1 000, 10 000 y 100 000 elementos. Para cada tamaño se generaron cinco patrones de datos, con el propósito de observar el comportamiento de los algoritmos ante distintas condiciones iniciales.

Para cada combinación de tamaño y patrón se realizaron 30 repeticiones. En cada repetición, MergeSort y QuickSort recibieron copias de la misma entrada original, garantizando que ambos procesaran los mismos valores. Asimismo, se utilizaron semillas para controlar la generación de los datos y se alternó el orden de ejecución de los algoritmos entre repeticiones para reducir posibles efectos asociados al orden de las pruebas.

En total, se realizaron 900 ejecuciones: dos algoritmos, tres tamaños, cinco patrones y 30 repeticiones. Los resultados se almacenaron en un archivo CSV para su posterior procesamiento estadístico.

2.4. Métricas de evaluación:

El rendimiento de los algoritmos se evaluó mediante cinco métricas:

1. Tiempo de ejecución (ms): tiempo empleado por el algoritmo para ordenar el arreglo, expresado en milisegundos.
2. Número de comparaciones: cantidad de comparaciones entre valores contabilizadas por los contadores implementados en cada algoritmo.
3. Número de movimientos: cantidad de operaciones de movimiento de elementos contabilizadas durante el ordenamiento. En MergeSort se incluyen las inserciones en el arreglo auxiliar y las copias de regreso al arreglo original; en QuickSort se contabilizan tres movimientos por cada intercambio efectuado.
4. Número de intercambios: cantidad de intercambios realizados entre elementos durante el ordenamiento. Esta métrica corresponde principalmente al procedimiento de partición de QuickSort; en MergeSort no se realizan intercambios de ese tipo.
5. Profundidad máxima de recursión: mayor nivel de profundidad alcanzado durante las llamadas recursivas del algoritmo, utilizado para observar las características de su proceso de división.

Los resultados se analizaron mediante medidas descriptivas, como la media, la mediana, la desviación estándar, el mínimo y el máximo. Además, se compararon los tiempos de ejecución de ambos algoritmos para las mismas entradas y se aplicaron pruebas estadísticas para evaluar las diferencias observadas.

3. Resultados

Esta sección presenta los resultados obtenidos durante la evaluación de MergeSort y QuickSort. Se utilizaron medidas de estadística descriptiva para resumir el tiempo de ejecución y las operaciones registradas, considerando tres tamaños de entrada y cinco patrones de datos.

3.1. Estadística descriptiva del tiempo de ejecución:

La Tabla 1 presenta el tiempo promedio y la desviación estándar de ambos algoritmos para arreglos de 100 000 elementos. Se utiliza este tamaño para facilitar la comparación de los resultados en una tabla compacta.

Tabla 1. Tiempo de ejecución para arreglos de 100 000 elementos

Patrón	MergeSort (media ± DE)	QuickSort (media ± DE)
Aleatorio	19,338 ± 1,154	10,212 ± 0,516
Ascendente	10,448 ± 0,817	1,251 ± 0,218
Descendente	9,910 ± 0,848	1,368 ± 0,154
Casi ordenado	10,852 ± 0,603	1,772 ± 0,239
Duplicados	14,582 ± 0,958	5,022 ± 0,542

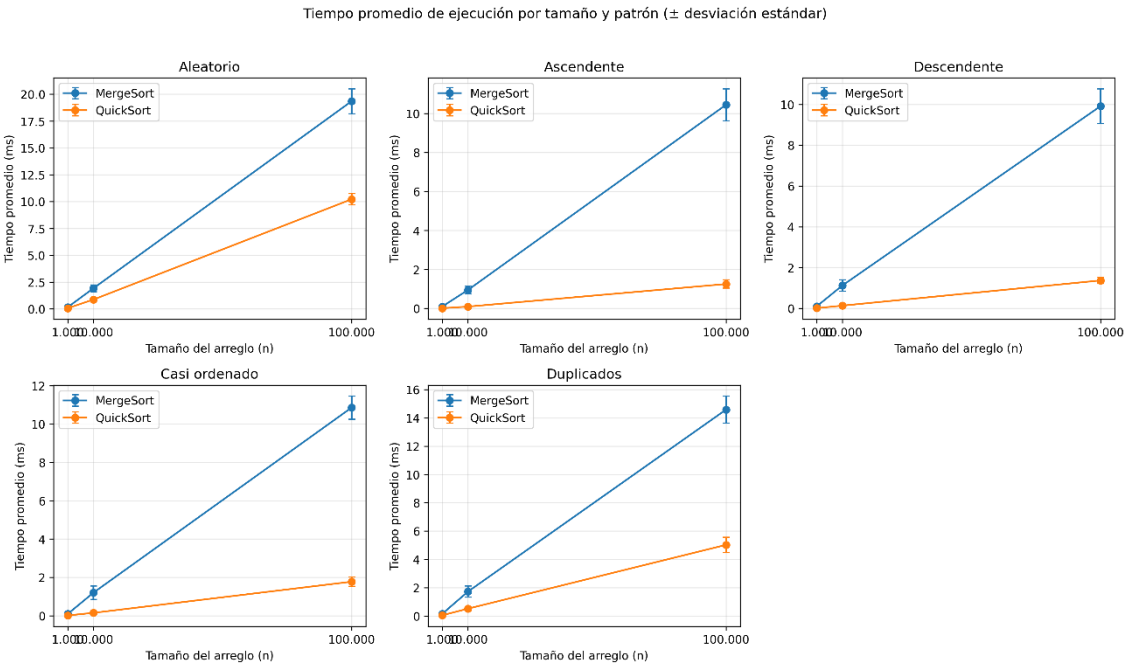
Nota. DE = desviación estándar. Los tiempos están expresados en milisegundos.

Los resultados muestran que QuickSort presentó un menor tiempo promedio en los cinco patrones analizados. La diferencia más marcada se observó en el patrón ascendente, mientras que en el patrón aleatorio la diferencia relativa fue menor. Además, la desviación estándar permite observar que los tiempos registrados variaron entre repeticiones, aunque la magnitud de esa variación dependió del algoritmo y del patrón de entrada.

3.2. Comparación gráfica del tiempo de ejecución:

La Figura 1 permite observar cómo varía el tiempo promedio al aumentar el tamaño del arreglo, diferenciando los cinco patrones de entrada. Las barras de error representan una desviación estándar por encima y por debajo de la media.

Figura 1. Tiempo promedio de ejecución



En la figura se observa que el tiempo de ejecución aumenta con el tamaño del arreglo para ambos algoritmos. Sin embargo, el incremento no es igual en todos los patrones. QuickSort presentó tiempos especialmente bajos para los arreglos ascendentes y descendentes, mientras que los arreglos aleatorios y los que contienen duplicados requirieron más tiempo.

3.3. Estadística descriptiva de las operaciones:

Además del tiempo, se analizaron las comparaciones, los movimientos, los intercambios y la profundidad máxima de recursión. La Tabla 2 resume las operaciones registradas para arreglos de 100 000 elementos.

Tabla 2. Promedio de operaciones para arreglos de 100 000 elementos

Patrón	Algoritmo	Comparaciones	Movimientos	Intercambios
Aleatorio	MergeSort	1 536 316	3 337 856	0
Aleatorio	QuickSort	2 251 025	1 160 311	386 770
Ascendente	MergeSort	853 904	3 337 856	0
Ascendente	QuickSort	1 600 016	0	0
Descendente	MergeSort	815 024	3 337 856	0
Descendente	QuickSort	1 600 030	150 000	50 000
Casi ordenado	MergeSort	1 404 374	3 337 856	0
Casi ordenado	QuickSort	1 616 958	44 250	14 750
Duplicados	MergeSort	1 512 444	3 337 856	0
Duplicados	QuickSort	1 727 477	1 971 331	657 110

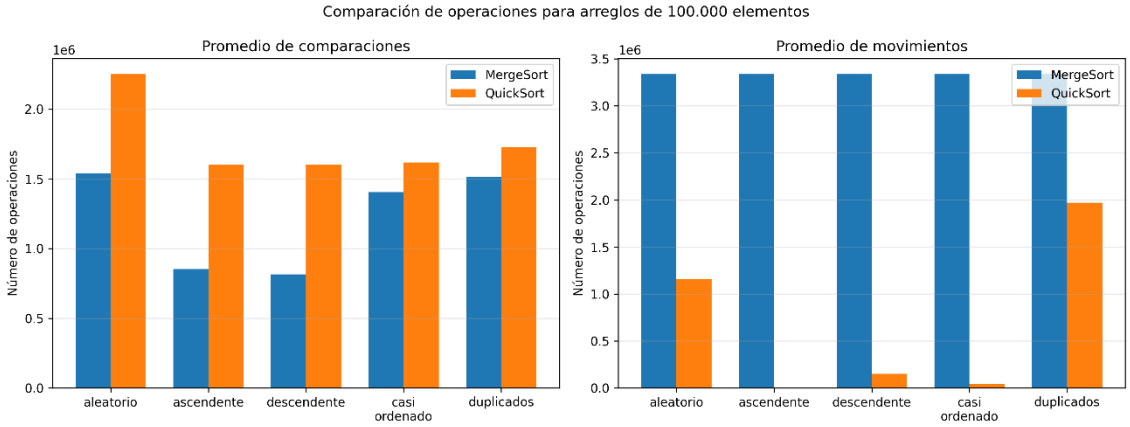
Nota. Los valores representan promedios de 30 repeticiones, redondeados al entero más cercano.

En los patrones analizados, MergeSort registró menos comparaciones que QuickSort, pero contabilizó más movimientos. Esto se relaciona con su procedimiento de combinación, que utiliza un arreglo auxiliar. QuickSort, en cambio, presentó una cantidad de movimientos que varió considerablemente según el patrón de entrada.

En el caso ascendente, QuickSort no registró movimientos ni intercambios porque la estrategia de partición utilizada pudo procesar el arreglo sin intercambiar elementos. Esto no significa que no haya realizado trabajo: continuó efectuando comparaciones y llamadas recursivas.

3.4. Comparación gráfica de las operaciones:

Figura 2. Comparación de operaciones para arreglos de 100 000 elementos



La figura permite comparar las diferencias entre el número de comparaciones y movimientos de ambos algoritmos. En particular, muestra que QuickSort puede realizar más comparaciones y, aun así, registrar un menor tiempo de ejecución. Por tanto, ninguna métrica aislada explica por completo el rendimiento observado.

3.5. Archivos con las tablas completas:

Los archivos CSV con los datos estadísticos descriptivos de todas las combinaciones, no solamente las correspondientes a 100 000 elementos se encuentran en el repositorio de GitHub.

Estos archivos pueden utilizarse para elaborar tablas adicionales o comprobar las cifras incluidas en el artículo.

4. Discusión:

Interpretación frente a la complejidad algorítmica. Se puede ver en los resultados experimentales que el rendimiento de los algoritmos MergeSort y QuickSort tiene que ver tanto con el tamaño del arreglo como con cómo están inicialmente distribuidos los elementos. Además de esto también influyen las particularidades de cada implementación. De esta manera, se puede apreciar cómo ambos algoritmos tienen un comportamiento de $O(n \log n)$ en condiciones promedio. Sin embargo, sus tiempos de ejecución son distintos debido a que realizan distintas cantidades de comparaciones,

movimientos de elementos y operaciones internas. En este experimento se pueden apreciar las diferencias entre ambos métodos. De esta forma se ve que aunque la complejidad teórica ayuda a entender el comportamiento global de los algoritmos no es suficiente para determinar cuál de ellos tiene menor tiempo de ejecución en una situación concreta.

Estabilidad de los algoritmos. Otro aspecto a tener en cuenta en la comparativa entre los diferentes métodos de ordenación es la estabilidad de estos. En el caso de MergeSort, existe la posibilidad de mantener el orden relativo de aquellos elementos que tienen la misma clave (siempre que en la fase de combinación de sublistas se privilegie el elemento de la izquierda). Esto no suele ser garantizado en el caso de QuickSort. Este aspecto, aunque importante, no fue objeto de estudio en este experimento (el cual estaba focalizado sobre todo en el tiempo de ejecución y las operaciones que se llevaban a cabo).

Limitaciones de la pila y recursividad. Ambos algoritmos emplean un esquema recursivo. Esto implica que se necesita un gasto adicional de memoria para manejar las llamadas recursivas. En el caso de MergeSort la profundidad de las llamadas recursivas es generalmente proporcional a $\log n$ mientras que en QuickSort puede llegar a $O(n)$ en caso de que las particiones sean muy asimétricas. Esto puede llevar a un mayor gasto de memoria en la pila (y en casos extremos a un desbordamiento). Por otro lado, en la implementación de QuickSort hemos usado como pivote el elemento central. Esto puede ayudar en algunos patrones de entrada a obtener particiones más equilibradas. De hecho podemos ver en la profundidad máxima alcanzada en nuestro experimento este comportamiento. Cabe destacar que esto no quiere decir que el total de memoria consumida sea esta cantidad o que hayamos llegado al límite de la pilla.

Amenazas a la validez. Es importante tener en cuenta algunas restricciones de este experimento al momento de interpretar los resultados. La medición se hizo en un ambiente de ejecución particular. Por esto los tiempos pueden variar en otros equipos con otro tipo de procesador o cantidad de memoria o sistemas operativo. Aunque se hicieron varias repeticiones y se probaron distintos tamaños y 5 patrones de entrada, las conclusiones sólo son válidas para los casos analizados e implementaciones utilizadas. El tiempo de ejecución puede verse influenciado por procesos en segundo plano u otros factores de sistema. Los contadores de comparaciones y movimientos están relacionados con su definición en el código.

5. Conclusiones

1. Rendimiento: La comparación experimental permitió identificar diferencias de rendimiento entre MergeSort y QuickSort según el tamaño y el patrón de los datos, demostrando que la eficiencia práctica depende tanto del algoritmo como de su implementación.

2. Complejidad algorítmica: Aunque ambos algoritmos presentan un comportamiento promedio de $O(n \log n)$, sus tiempos de ejecución y cantidades de operaciones pueden

variar, por lo que la complejidad teórica debe complementarse con mediciones experimentales.

3. Estabilidad y recursión: MergeSort puede mantener el orden relativo de elementos iguales, mientras que QuickSort generalmente no garantiza estabilidad. Además, la profundidad recursiva de ambos algoritmos influye en el uso de memoria, especialmente cuando QuickSort genera particiones desequilibradas.

4. Alcance del experimento: Las pruebas realizadas con distintos tamaños, patrones y repeticiones permitieron comparar ambos métodos bajo condiciones definidas; sin embargo, los resultados corresponden a las implementaciones y al entorno evaluados, por lo que no deben generalizarse a todas las situaciones.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). The MIT Press.

Hoare, C. A. R. (1962). Quicksort. *The Computer Journal*, 5(1), 10–16. https://doi.org/10.1093/comjnl/5.1.10

Cohen, C., & Sakaguchi, K. (2025). A bargain for mergesorts: How to prove your mergesort correct and stable, almost for free. Proceedings of the ACM on Programming Languages, 9(ICFP), Article 236. https://doi.org/10.1145/3747505

Moret, B. M. E. (2002). Algorithms and experiments: The new (and old) methodology. Journal of Universal Computer Science, 8(1), 1–14.